StitchCraft 三方研究综合报告

日期：2026-03-15

三方：Claude Opus（知识工程+产品架构）/ Gemini（产品设计+用户体验）/ Codex（工程实现+算法）

一、三方最核心共识

共识 1：缝合 ≠ 自动拼装，缝合 = 带约束的知识组合优化

• Claude：排除了合并/编译/蒸馏/生成四个候选模型，确定缝合是"在已验证知识对象中按场景约束寻找互补组合的帕累托最优前沿"

• Codex："第一版不应该做代码缝合怪生成器，而应该做知识层缝合编译器"。明确三层分离：Knowledge Stitch → Architecture Stitch → Code Graft，MVP 只到前两层

• Gemini：虽然描述了更激进的"器官移植"交互，但核心也是"交互式引导"而非全自动

结论：三方一致——缝合的输出是"组合可能性地图"，不是"缝好的 skill"。 Doramagic 给 X 光片，不开处方。

共识 2：冲突不应被消解，冲突本身是高价值知识

• Claude：三类冲突分别处理——事实冲突确定性解决，scope 冲突标注 scope 保留双方，路线冲突展示帕累托前沿让用户决策

• Codex：6 类冲突（semantic/scope/architecture/dependency/operational/license），4 种解决策略（Preserve Both / Promote Consensus / Ask User / Block）

• Gemini："陪审团"模式展示冲突，将冲突转化为选择题

结论：冲突密度是领域成熟度的元指标。SEO 权重分配的路线之争（GEO-first vs Technical-first）不是 bug，是 feature。

共识 3：建立在已有架构之上，增量开发

• Claude：缝合完全建立在 fingerprint + compare_projects 之上，是跨项目智能管线的下游消费者

• Codex：最小新增 4 个组件（stitch_profile.json / stitch_candidates.json / conflict_graph.json / stitch_compiler.py）

• Gemini：输出应是 OpenClaw Skill Package（对接已有生态）

结论：不需要新管线。在已规划的 project_fingerprint.json + compare_projects.py 上叠加缝合层，增量 2-3 天。

共识 4：License 合规是硬门控

• Claude：缝合地图标注每个 capability 的源项目 license，credit 是强制的

• Codex：最详细——SPDX expression 机器化处理、MIT/Apache/GPL 兼容性规则、knowledge_only 和 code_graft 两种模式、license_gate 硬阻断

• Gemini：强制生成 CONTRIBUTORS_STITCHED.md

结论：第一版只做知识层缝合（低 license 风险），不做代码层 graft。每个知识对象强制带 source_project + license 标注。

二、各方独有最佳贡献

Claude 独有：理论深度

1. 四个候选模型的排除推导（合并→不同构；编译→仅确定性子集；蒸馏→消解分歧=摧毁信息；生成→双重违规）——这个排除法是缝合设计的理论基石

2. 帕累托最优前沿：缝合不选择最优，展示帕累托前沿。具体到 4 个 SEO 项目：组合 A（GEO 优先）/ 组合 B（技术全面）/ 组合 C（最大覆盖），各有适用场景

3. 甜蜜区间定义：N=3-8 个项目、40-70% 共识、有明确场景约束、领域正在转型。4 个 SEO 项目恰好落在甜蜜区间

4. 其他领域类比：药物联合治疗（禁忌症→冲突图）/ 音频混音（频段分离→层次化知识）/ 法律判例综合（ratio decidendi vs obiter dictum）/ 系统生物学通路分析（合成致死→组合后新问题）

5. 更大可能性：积累足够项目后，缝合地图演变为领域知识地图（行业共识/前沿争论/过时知识/被低估独创）+ 知识时效监控 + 作业推荐引擎

Gemini 独有：产品直觉

1. 三类核心用户：架构选型者（The Architect）/ 领域专家（The Domain Specialist）/ AI Agent 开发者（The Agent Builder）

2. "缝合四部曲"交互设计：零件拆解 → 基座选择 → 器官移植 → 灵魂融合。虽然比 Claude/Codex 更激进（接近代码层），但用户旅程设计最具体

3. 商业价值判断：缝合是 Doramagic 从"阅读器"变成"生产力工具"的关键跃迁。增值服务方向：企业级灵魂缝合（内部私有知识 + 开源最佳实践）

4. "缝合提案"主动推送：Doramagic 检测到用户在研究 SEO 领域，主动推送"全明星缝合版"对比报告

5. 类比：Doramagic = 代码界的 Splice（音乐采样平台）

6. 知识"排异反应"风险：A 项目哲学"极简"，B 项目"冗余防御"，缝合后 CLAUDE.md 可能产出矛盾指令。需要"哲学一致性检查"

Codex 独有：工程深度

1. 完整伪代码：`stitch_projects()` 主函数 + `compile_stitch_profile()` + `extract_consensus_layers()` + `extract_unique_grafts()` + `detect_conflicts()` + `resolve_conflict()` + `validate_stitched_package()` — 可直接作为实现参考

2. 冲突检测公式：`ConflictScore = claim_opposition × scope_overlap × implementation_incompatibility × normative_strength`

3. Stitch Validation Gate（Stage S）：6 项检查（Consistency / Dependency closure / Assumption closure / License closure / Traceability / Actionability），输出 PASS/REVISE/BLOCKED

4. License 工程化：SPDX expression 规范化 + `license-expression` Python 库 + Apache/GPL 兼容性规则 + knowledge_only / code_graft 双模式

5. 可借鉴技术栈：

- Knowledge Fusion survey（多源冲突 + truth inference）
- mergekit（模型合并的 slice-wise assembly 思想 → knowledge mergekit）
- SPDX + license-expression 库

6. 9 步数据流：fingerprint → stitch_profile → canonicalize → cluster → consensus → uniqueness → conflicts → foundation selection → validate → compile outputs

7. 关键工程警告：

- "能力能缝"不等于"依赖能缝"——capability 必须带 required_files/hooks/commands/env
- 文档 license 比代码更混乱——只输出重新表述，不长段搬运原文
- Quality Gate 要防"伪统一叙事"——LLM 会把冲突编成漂亮故事
- UI 从"组合工作台"开始，不从"融合按钮"开始
- 评测集是门槛（3 类 gold set：可安全组合 / 强冲突 / license 不兼容）

三、三方分歧与决策建议

分歧 1：用户参与深度

- Claude：用户看 STITCH_MAP.md，自己决策（最克制）
- Gemini：用户交互式"器官移植"，拖拽选择组件（最激进）
- Codex：用户看组合工作台，选骨架、选 graft、看冲突（中间路线）

决策建议：V1 采 Claude + Codex 路线（输出报告 + 工作台），V2 考虑 Gemini 的交互式移植。先做对的再做酷的。

分歧 2：输出形态

- Claude：STITCH_MAP.md（纯报告）
- Gemini：OpenClaw Skill Package（可安装产物）
- Codex：4 个文件（STITCH_REPORT.md + STITCHED_KNOWLEDGE.md + ASSEMBLY_PLAN.md + LICENSE_NOTICE.md）

决策建议：采 Codex 的 4 文件输出（最工程化、最可审计）。Gemini 的 Skill Package 是 V2 目标。

分歧 3：主动推送 vs 被动触发

- Gemini：Doramagic 主动推送"缝合提案"（"检测到你在研究 SEO 领域..."）
- Claude/Codex：用户主动触发

决策建议：V1 被动触发（用户丢 N 个链接进来）。主动推送需要积累足够领域数据后才有价值，是 V2+ 功能。

四、完整实施路径

Phase 0：前置依赖（已规划）

- project_fingerprint.json schema 定稿
- compare_projects.py 可运行

Phase 1：Stitch Profile（1 天）

- 从 fingerprint 生成 stitch_profile.json（capabilities / decisions / patterns / constraints / assumptions / licenses）
- 每个对象标注 uniqueness（SHARED/UNIQUE/VARIANT）+ conflicts_with 边

Phase 2：冲突检测 + 共识提取（1-2 天）

- conflict_graph.json（6 类冲突检测）
- consensus_layer.json（含 provenance clustering 防假公约数）
- unique_grafts.json（独创贡献标注 graftable/reference_only/experimental）

Phase 3：Stitch Compiler + Validation Gate（1-2 天）

- stitch_compiler.py → 4 个输出文件
- Stitch Validation Gate（6 项检查）
- license_gate（knowledge_only 模式）

Phase 4：验证（1 天）

- 用 4 个 SEO 项目做第一个 gold set
- 人工验证共识/独创/冲突/组合建议

总计：4-6 天增量开发（在 fingerprint + compare 就绪后）

五、一句话总结

> StitchCraft 不是自动裁缝，是制版师。它给你裁剪图、面料清单和禁忌症标注，你决定怎么缝。这恰好是"不教用户做事，给他工具"的完美体现。